РУКОВОДСТВО СИСТЕМНОГО ПРОГРАММИСТА

Симулятор лифта с алгоритмом SCAN

Версия 1.0

АННОТАЦИЯ

Настоящий документ представляет собой руководство системного программиста для программного комплекса «Симулятор лифта с алгоритмом SCAN».

Документ содержит сведения, необходимые для:

- Понимания архитектуры программного обеспечения;
- Настройки параметров работы системы;
- Сопровождения и модификации исходного кода;
- Диагностики и устранения неисправностей.

Документ предназначен для специалистов, обладающих знаниями языка программирования Python и принципов объектно-ориентированного программирования.

Оглавление

1. ОБЩИЕ СВЕДЕНИЯ О ПРОГРАММЕ 3

1.1. Назначение программы 3

1.2. Функциональные возможности 3

1.3. Технические характеристики 3

2. СТРУКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ 3

2.1. Состав программных модулей 3

2.2. Описание классов и их назначение 4

2.3. Схема взаимодействия модулей 6

3. НАСТРОЙКА И КОНФИГУРИРОВАНИЕ 6

3.1. Параметры моделирования 6

3.2. Изменение количества этажей 7

4. ПРОВЕРКА РАБОТОСПОСОБНОСТИ 7

4.1. Запуск программы 7

4.2. Диагностические сообщения 8

4.3. Типовые неисправности и способы их устранения 8

5. СООБЩЕНИЯ СИСТЕМНОМУ ПРОГРАММИСТУ 9

5.1. Логика конечного автомата 9

5.2. Реализация алгоритма SCAN 10

5.3. Особенности визуализации 10

ПРИЛОЖЕНИЕ А. ТЕКСТЫ ДИАГНОСТИЧЕСКИХ СООБЩЕНИЙ 11

1. ОБЩИЕ СВЕДЕНИЯ О ПРОГРАММЕ

1.1. Назначение программы

Программный комплекс «Симулятор лифта с алгоритмом SCAN» предназначен для:

- Моделирования алгоритма обслуживания запросов SCAN («Лифт»);
- Визуализации работы конечного автомата управления лифтом;
- Демонстрации принципов диспетчеризации в системах реального времени.

1.2. Функциональные возможности

Программа обеспечивает выполнение следующих функций:

- Прием и обработку внешних вызовов с этажей (направление вверх/вниз);
- Прием и обработку внутренних вызовов (выбор этажа назначения);
- Формирование очереди запросов согласно алгоритму SCAN;
- Имитацию движения кабины с заданной скоростью;
- Имитацию цикла открытия/закрытия дверей;
- Визуализацию текущего состояния системы в графическом интерфейсе.

1.3. Технические характеристики

- Язык программирования: Python 3.8+
- Графическая библиотека: Tkinter (стандартная)
- Используемые модули: enum, time, collections.deque
- Объем исходного кода: ~400 строк
- Режим работы: интерактивный, событийно-ориентированный
- Частота обновления анимации: 20 кадров/сек (50 мс)

2. СТРУКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1. Состав программных модулей

Программный комплекс состоит из одного файла исходного кода:

- main.py — основной исполняемый файл, содержащий всю логику работы симулятора и графический интерфейс пользователя.

Дополнительные файлы, входящие в дистрибутив:

- README.md — инструкция по развертыванию;

- Руководство_пользователя.pdf — эксплуатационная документация;

- Руководство_системного_программиста.pdf — настоящий документ.

2.2. Описание классов и их назначение

В состав программы входят следующие классы (Таблица 1):

Таблица 1 — Перечень классов программы

Класс	Назначение
Direction (Enum)	Перечисление направлений движения
ElevatorState (Enum)	Перечисление состояний конечного автомата
Elevator	Ядро логики симуляции, реализация алгоритма
ElevatorVisualization	Графический интерфейс и визуализация

2.2.1. Класс Direction (Enum)

Наследуется от enum.Enum. Определяет возможные направления движения лифта:

- UP = 1 — движение вверх;

- DOWN = -1 — движение вниз;

- IDLE = 0 — отсутствие движения.

2.2.2. Класс ElevatorState (Enum)

Наследуется от enum.Enum. Определяет состояния конечного автомата лифта:

- IDLE = "Ожидание" — лифт неподвижен, запросы отсутствуют;

- MOVING = "Движение" — кабина перемещается между этажами;

- DOOR_OPEN = "Двери открыты" — двери открыты, посадка/высадка пассажиров;

- DOOR_CLOSING = "Закрытие дверей" — переходное состояние перед движением.

2.2.3. Класс Elevator

Основной класс, реализующий логику работы лифта.

Атрибуты класса:

- num_floors: int — общее количество этажей в здании;
- current_floor: int — текущий этаж;
- target_floor: int | None — целевой этаж;
- direction: Direction — текущее направление движения;
- state: ElevatorState — текущее состояние конечного автомата;
- external_requests: set — множество внешних вызовов (этаж, направление);
- internal_requests: set — множество внутренних вызовов (этаж);
- requests_queue: deque — очередь запланированных этажей;
- door_open_time: float — время удержания дверей открытыми (сек);
- floor_travel_time: float — время проезда одного этажа (сек).

Методы класса:

- add_external_request(floor, direction) — добавление внешнего вызова;
- add_internal_request(floor) — добавление внутреннего вызова;
- update_requests_queue() — перестроение очереди по алгоритму SCAN;
- update(dt) — обновление состояния (вызывается каждый кадр).

2.2.4. Класс ElevatorVisualization

Класс, отвечающий за создание и управление графическим интерфейсом.

Атрибуты класса:

- num_floors: int — количество этажей;
- elevator: Elevator — экземпляр модели лифта;
- root: tk.Tk — корневое окно Tkinter;
- canvas: tk.Canvas — холст для отрисовки шахты;
- floor_frames: list — список панелей этажей;
- up_buttons: dict — кнопки вызова «вверх»;

- down_buttons: dict — кнопки вызова «вниз»;

- inside_buttons: dict — кнопки внутри кабины;

- info_label: tk.Label — метка с информацией о состоянии;

- queue_label: tk.Label — метка с очередью запросов.

Методы класса:

- call_elevator(floor, direction) — обработчик внешнего вызова;

- press_inside_button(floor) — обработчик внутреннего вызова;

- draw_elevator() — отрисовка шахты и кабины;

- update_ui() — обновление текстовых меток и индикаторов;

- update() — главный цикл анимации;

- run() — запуск главного цикла Tkinter.

2.3. Схема взаимодействия модулей

Взаимодействие компонентов программы организовано по шаблону Model-View:

1. Пользователь взаимодействует с элементами GUI (ElevatorVisualization);
2. GUI вызывает методы объекта Elevator (add_external_request, add_internal_request);
3. Elevator обрабатывает запрос и обновляет внутреннее состояние;
4. В каждом кадре анимации вызывается Elevator.update(dt), который управляет конечным автоматом;
5. После обновления состояния GUI перерисовывает кабину и индикаторы.

3. НАСТРОЙКА И КОНФИГУРИРОВАНИЕ

3.1. Параметры моделирования

Параметры моделирования задаются в исходном коде программы. Изменение параметров не требует перекомпиляции, достаточно отредактировать файл main.py.

Основные настраиваемые параметры приведены в Таблице 2.

Таблица 2 — Настраиваемые параметры

Параметр	Расположение в коде	Значение по умолчанию
Количество этажей	ElevatorVisualization(...)	10
Время открытия дверей	Elevator.door_open_time	3.0 (сек)
Время проезда этажа	Elevator.floor_travel_time	1.0 (сек)
Частота анимации	root.after(50, ...)	50 мс

3.2. Изменение количества этажей

Для изменения количества этажей в здании необходимо:

1) Открыть файл main.py в текстовом редакторе;

2) Найти блок кода в конце файла:

```
if __name__ == "__main__":
    # Создаем и запускаем визуализацию
    app = ElevatorVisualization(num_floors=10)
    app.run()
```

3) Заменить значение параметра num_floors на требуемое (например, 15);

4) Сохранить файл и перезапустить программу.

3.3. Настройка временных интервалов

Временные интервалы задаются в конструкторе класса Elevator:

- door_open_time — время в секундах, в течение которого двери остаются открытыми после прибытия на этаж;
- floor_travel_time — время в секундах, затрачиваемое на перемещение между соседними этажами.

Для изменения значений отредактируйте соответствующие строки в методе __init__() класса Elevator.

4. ПРОВЕРКА РАБОТОСПОСОБНОСТИ

4.1. Запуск программы

Для запуска программы выполните следующие действия:

1) Откройте терминал (командную строку) в директории с файлом main.py;

2) Выполните команду:

python main.py

3) Убедитесь в появлении графического окна приложения.

4.2. Диагностические сообщения

В процессе работы программа выводит диагностические сообщения в стандартный поток вывода (stdout/консоль). Сообщения позволяют отслеживать:

- Поступление внешних вызовов;
- Нажатие кнопок внутри кабины;
- Начало движения к целевому этажу;
- Прибытие на этаж и открытие дверей;
- Закрытие дверей.

Пример вывода диагностических сообщений:

Вызов с этажа 5 в направлении UP

Нажата кнопка этажа 8 внутри лифта

Начинаю движение к этажу 5

Прибыл на этаж 5, открываю двери

Закрываю двери

Полный перечень диагностических сообщений приведен в Приложении А.

4.3. Типовые неисправности и способы их устранения

Таблица 3 — Типовые неисправности

Неисправность	Вероятная причина	Способ устранения
Ошибка «python is not recognized»	Python не установлен или не добавлен в PATH	Установить Python, добавить в PATH при установке
Ошибка «No module named 'tkinter'»	Tkinter не установлен (Linux)	Выполнить: sudo apt-get install python3-tk
Графическое окно не отображается	Проблемы с дисплейным сервером (Linux)	Проверить настройки X11/Wayland

Кнопки не реагируют на нажатия	Программа в состоянии MOVING или DOOR_OPEN	Дождаться открытия дверей или состояния IDLE
Анимация работает рывками	Высокая нагрузка на CPU	Увеличить задержку в root.after

5. СООБЩЕНИЯ СИСТЕМНОМУ ПРОГРАММИСТУ

5.1. Логика конечного автомата

Управление лифтом реализовано на основе конечного автомата (Finite State Machine) с четырьмя состояниями. Переходы между состояниями осуществляются в методе Elevator.update(dt) (Таблица 4).

Таблица 4 — Матрица переходов конечного автомата лифта

Текущее состояние	Событие/Условие	Следующее состояние	Действие
IDLE	В очереди есть запросы	MOVING	Установить target_floor, определить direction
MOVING	Достигнут target_floor	DOOR_OPEN	Сбросить таймер дверей, удалить обслуженные запросы
MOVING	Достигнут промежуточный этаж (внутренний запрос)	DOOR_OPEN	Сбросить таймер дверей
DOOR_OPEN	Таймер >= door_open_time	DOOR_CLOSING	Обновить очередь запросов
DOOR_CLOSING	В очереди есть запросы	MOVING	Установить target_floor, определить direction
DOOR_CLOSING	Очередь пуста	IDLE	Сбросить direction в IDLE

ВАЖНО: при модификации кода не рекомендуется изменять порядок проверки состояний в методе update(), так как это может привести к пропуску кадров анимации или некорректной обработке запросов.

5.2. Реализация алгоритма SCAN

Алгоритм SCAN («Лифт») реализован в методе Elevator.update_requests_queue().

Логика работы алгоритма:

1) Сбор всех активных запросов:

- Все внутренние запросы (internal_requests);
- Внешние запросы, направление которых совпадает с текущим направлением движения лифта.

2) Сортировка запросов:

- При движении ВВЕРХ: сначала все этажи >= текущего (по возрастанию), затем все этажи < текущего (по убыванию);
- При движении ВНИЗ: сначала все этажи <= текущего (по убыванию), затем все этажи > текущего (по возрастанию).

3) Запись отсортированной последовательности в requests_queue.

4) Удаление обработанных запросов происходит при открытии дверей на соответствующем этаже.

При внесении изменений в алгоритм следует учитывать, что requests_queue должна быть экземпляром collections.deque для эффективного извлечения первого элемента.

5.3. Особенности визуализации

Визуализация выполняется средствами Tkinter Canvas. Ключевые особенности:

1) Размеры холста пересчитываются при каждом кадре (метод draw_elevator);

2) Если размер окна меньше 100x100 пикселей, отрисовка пропускается;

3) Координаты кабины вычисляются на основе текущего этажа и высоты холста;

4) Двери отрисовываются по-разному в зависимости от состояния ElevatorState.DOOR_OPEN.

Для повышения производительности рекомендуется:

- Не изменять частоту обновления без необходимости;
- Избегать создания новых объектов Tkinter в цикле анимации;
- Использовать self.canvas.delete("all") для очистки холста перед перерисовкой.

ПРИЛОЖЕНИЕ А. ТЕКСТЫ ДИАГНОСТИЧЕСКИХ СООБЩЕНИЙ

Таблица А.1 — Диагностические сообщения программы

Сообщение	Условие вывода
Вызов с этажа N в направлении UP/DOWN	Добавлен внешний запрос
Нажата кнопка этажа N внутри лифта	Добавлен внутренний запрос
Начинаю движение к этажу N	Переход в состояние MOVING
Прибыл на этаж N, открываю двери	Достижение целевого этажа
Остановка на этаже N по внутреннему вызову	Промежуточная остановка
Закрываю двери	Переход в состояние DOOR_CLOSING

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Изм.	Номера листов (страниц)			Всего листов в док.	Входящий № сопровод. док-та и дата	Подп.	Дата
	измененных	замененных	новых				
1	-	-	Все	11	Первая редакция		

КОНЕЦ ДОКУМЕНТА